Prompt evaluator quick reference

Setup

```
conda create -n prompt-evaluator python=3.11
conda activate prompt-evaluator
python -m pip install -r requirements.txt
# Optional Parquet engine:
python -m pip install -r requirements-parquet.txt
python validate_setup.py
```

Activate an existing environment instead of creating it again. Default validation is offline. Configure the confirmed tenant with `CALYPSOAI_URL=https://us1.calypsoai.app` (or EU1/custom origin) and `CALYPSOAI_TOKEN`; environment values override `.env`.

Live evaluation

Confirm the tenant/project and start with synthetic data. These commands send input to the API:

```
python prompt_evaluator.py -i datasets/sample-datasets/pii_dataset.jsonl -l 10 --project \
  evaluation-project
python prompt_evaluator_prompts.py -i datasets/sample-datasets/pii_dataset.jsonl -m 10 \
  --project evaluation-project --provider evaluation-provider --results-format json
```

Both accept `--timeout`, `--verbose`, `--diagnostics`, `--format-hint`, `--output`, `--results-format`, and `--format`. Prompts enables verbose by default and is the only mode with provider selection. Concurrent scans use `-c` and `--rate-limit` after infrastructure approval and interactive confirmation.

Main output defaults: sequential/concurrent scans use JSONL; prompts uses CSV. FP/FN exports default to JSONL in every mode. All outputs live in `results/`. Custom main filenames must match the selected format and cannot collide with inputs/exports.

Scoring

Raw outcomes stay unchanged. Flagged, blocked, and redacted count as detections; cleared counts as no detection. Accuracy is `(TP + TN) / evaluated`; precision is `TP / (TP + FP)`; recall is `TP / (TP + FN)`; F1 is their harmonic mean.

Always inspect attempted, evaluated, error, incomplete, and invalid counts. Provider failures are separate. No evaluable rows means no accuracy estimate. Aggregate prompt-pipeline outcomes aren't individual-scanner accuracy. Historical opaque scanner cells can't establish execution completeness.

Offline analysis

```
python evaluate_existing_results.py -i results/pii_dataset_results.jsonl
python report_generator.py --dataset pii_dataset
python report_generator.py --input results/custom.jsonl
```

Sequential scans and prompts generate PDFs automatically; concurrent scans don't. `make test-concurrent` is live evaluation, not an offline test.

Results and PDF examples can contain PII. Additional content-rich diagnostics require `--diagnostics`; review access, retention, and content before sharing.

Development checks

```
python -m pip install -r requirements-test.txt
make test-core
make test-contract
```

Run these inside the project Conda environment. Optional Parquet cases require its engine. Generated results stay excluded from Git; only an empty directory placeholder is tracked.

See usage, metrics, reports, and troubleshooting.